

Assignment 2: Data Ingestion & Data Warehouse Modelling

Onboarding Flow Optimization A/B Test

Aishwarya Mishra

August 2026

Table of Contents

Abstract

1. Introduction

2. Architectural Overview

2.1 Layer Summary

2.2 Rationale for Layering

2.3 Rationale for a Dimensional (Star Schema) Model

3. Layer-by-Layer Design

3.1 Raw Layer (`schema/01_raw.sql`)

3.2 Staging Layer (`schema/02_staging.sql`)

3.3 Warehouse Layer (`schema/03_warehouse.sql`)

3.4 Data Mart Layer (`schema/04_datamart.sql`)

4. Ingestion Design

5. Data Quality Framework

5.1 Verifying the Validation Logic Itself

6. Observed Results

7. Assumptions and Limitations

8. How to Run

Abstract

This document specifies the design and implementation of the data platform that transforms the raw event stream defined in Assignment 1 into an analysis-ready warehouse and data mart. It follows a four-layer (“medallion”) architecture — Raw, Staging, Warehouse, Data Mart — implemented in SQLite for portability, and documents the rationale behind each design decision: why this layering rather than a single flat table, why a dimensional (star-schema) model rather than a normalized transactional model, what the ingestion and deduplication contract is, and how data quality is verified rather than assumed. Every SQL artifact referenced here is version- controlled under `schema/` and executed, in order, by `build_db.py`.

1. Introduction

Assignment 1 produces an append-only, long-format event stream: one row per user action, timestamped, tagged with experiment arm and platform. That format is optimal for *capture* — it can represent any event type without a schema change — but is poor for *analysis*: computing a single KPI such as “onboarding completion rate by arm” against the raw event table requires a self-join and conditional aggregation that is expensive, error-prone to repeat correctly across every metric in the KPI framework, and opaque to a non-technical dashboard consumer. This assignment closes that gap.

2. Architectural Overview

2.1 Layer Summary

```
raw_events                                RAW
| (1) validate (2) deduplicate (3) flatten JSON -> typed columns
v
stg_events, stg_events_rejects            STAGING
| (4) dimensional modelling (star schema)
v
dim_user, dim_date, dim_step,
fact_onboarding_step_events,
fact_user_funnel                          WAREHOUSE
| (5) group-by aggregation to KPI/guardrail grain
v
mart_kpi_by_group, mart_guardrails_by_group,
mart_step_dropoff, mart_daily_funnel      DATA MART
```

2.2 Rationale for Layering

A single-layer design — transforming raw JSON events directly into the final KPI tables in one pass — was considered and rejected for three reasons:

1. **Debuggability.** When a KPI number looks wrong, a layered pipeline lets the analyst inspect intermediate state (did the row survive staging? does it appear in the warehouse fact table? does the mart aggregation match a manual `GROUP BY` against the fact table?) rather than re-deriving the entire transformation from raw JSON to find the defect.

2. **Reusability.** `fact_user_funnel` (Warehouse layer) is the single source of truth for every KPI and guardrail in the Assignment 1 KPI framework. Without it, each new mart table would need to independently re-derive funnel state from raw events, multiplying the surface area for transformation bugs and definitional drift between marts.
3. **Auditability of data quality.** Validation and deduplication belong at a single, well-defined boundary (Raw to Staging) rather than being re-implemented, potentially inconsistently, in every downstream query.

2.3 Rationale for a Dimensional (Star Schema) Model

The warehouse layer uses a **star schema** — a central fact table (`fact_user_funnel`) at a fixed grain, joined to descriptive dimension tables (`dim_user` , `dim_date` , `dim_step`) — rather than a fully normalized (3NF) transactional model. This follows standard data- warehousing practice (the Kimball dimensional-modeling approach) for analytical workloads, for two concrete reasons specific to this dataset:

- **Query simplicity for the consuming layer.** Every KPI in the Assignment 1 framework is a `GROUP BY test_group` aggregation over a small number of boolean/flag columns. A star schema with one wide fact table at “one row per user” grain makes this a single-table aggregation; a normalized model would require reconstructing that same row via joins across several transactional tables at query time, for every dashboard query.
- **Fixed, known grain.** Because the business question in Assignment 1 is stated entirely in terms of per-user funnel outcomes (did this user complete onboarding, activate, return, purchase), the natural grain of the primary fact table is unambiguous: one row per user. A normalized model optimizes for transactional write consistency, which is not the workload here — the data is immutable, append-only event history.

The star schema is not applied dogmatically: a second, more granular fact table (`fact_onboarding_step_events` , grain: one row per step-level event) is retained alongside the summary fact, because step-level drop-off analysis (used by `mart_step_dropoff`) genuinely needs event-level, not user-level, granularity. This is a deliberate two-fact-table design rather than forcing a single grain to serve both purposes.

3. Layer-by-Layer Design

3.1 Raw Layer (`schema/01_raw.sql`)

`raw_events` is a column-for-column mirror of the source event CSV, plus two ingestion-metadata columns (`_ingested_at` , `_source_file`). No casting, filtering, or validation happens here — the raw layer’s sole responsibility is to be a faithful, replayable copy of what ingestion received, including any malformed rows, so that a staging-layer bug can always be diagnosed by re-running staging against an unmodified raw capture rather than needing to re-ingest from source.

Indexes on `event_id` and `user_id` are created at this layer purely to keep the staging deduplication query (Section 3.2) performant; they do not imply any constraint enforcement, which is intentionally deferred to staging.

3.2 Staging Layer (`schema/02_staging.sql`)

Staging performs three distinct transformations, applied in this order to avoid wasted work (validation and deduplication happen before the more expensive JSON-flattening step):

1. **Validation.** Rows failing any rule in Assignment 1's Section 6 (`event_id / user_id / event_name / event_timestamp` non-null; `test_group` in {A,B,C}; `platform` in {iOS,Android}) are copied, with a `reject_reason`, into `stg_events_rejects` — a quarantine table, not a silent drop. This is a deliberate choice: silently dropping bad rows makes data-quality regressions invisible; a populated `stg_events_rejects` table is itself a monitorable signal (Section 5).
2. **Deduplication.** Rows are windowed by `event_id` (`ROW_NUMBER() OVER (PARTITION BY event_id ORDER BY _ingested_at)`) and only the first-ingested copy of each `event_id` is kept. This makes staging idempotent under at-least-once delivery, a standard assumption for client-side event collection (a network retry after an unacknowledged write is expected to produce a duplicate `event_id`, not a new one).
3. **Typed flattening.** `event_timestamp` is cast from text to a real SQLite datetime, and the fifteen most commonly queried `properties` keys are extracted into typed columns via `json_extract(...)`, so that no downstream SQL (warehouse or mart layer) needs to parse JSON. Properties not flattened here remain accessible in the retained raw `properties` column for ad hoc analysis.

3.3 Warehouse Layer (`schema/03_warehouse.sql`)

Table	Grain	Purpose
<code>dim_user</code>	1 row per <code>user_id</code>	Arm, platform, install date, account-creation status
<code>dim_date</code>	1 row per calendar date in the observed range	Generated via a recursive CTE; supports time-series joins for <code>mart_daily_funnel</code>
<code>dim_step</code>	1 row per (arm, step position)	Loaded directly from <code>stepmap.json</code> rather than derived from events, since it is experiment-design metadata, not observed behavior
<code>fact_onboarding_step_events</code>	1 row per step-level event (viewed/completed/abandoned)	Granular fact for step-level drop-off analysis
<code>fact_user_funnel</code>	1 row per <code>user_id</code>	The primary analytical fact; every KPI and guardrail in Assignment 1 is a <code>GROUP BY test_group</code> aggregation over this single table

`fact_user_funnel` is constructed with one `LEFT JOIN` from `dim_user` to `stg_events`, pivoting the long event stream into wide, typed boolean and timestamp columns via `MAX(CASE WHEN event_name = '...' THEN 1 ELSE 0 END)` idioms. This pivot is the single point in the pipeline where “did this user do X” becomes a queryable column rather than a row that must be searched for; every mart table and every future dashboard query builds on this fact rather than re-deriving it.

Indexing strategy. `dim_user.user_id` carries a unique index (it is the fact table's join key); `fact_user_funnel.test_group` carries a non-unique index, since every mart query filters or groups by it. No index is placed on low-selectivity boolean flag columns, where a full scan is cheaper than an index lookup at this data volume.

3.4 Data Mart Layer (`schema/04_datamart.sql`)

Materialized (not virtual/view) tables, each built as a `GROUP BY` over `fact_user_funnel` or `fact_onboarding_step_events`, at the exact grain a dashboard or statistical test needs:

Mart Table	Grain	Consumed By
<code>mart_kpi_by_group</code>	1 row per arm	Primary/secondary KPI reporting; also the SRM check (Section 3, Assignment 1 event-tracking doc)
<code>mart_guardrails_by_group</code>	1 row per arm	Guardrail reporting
<code>mart_step_dropoff</code>	1 row per (arm, step)	Funnel/drop-off visualization
<code>mart_daily_funnel</code>	1 row per (date, arm)	Trend visualization; also supports a pre/post novelty-effect check

Marts are built as physical tables rather than SQL views specifically so that Assignment 4's dashboard queries are $O(\text{rows in mart})$, not $O(\text{rows in fact table})$ — a deliberate pre-aggregation trade-off, appropriate because the mart tables are small (rows = number of arms, or arms x steps) and rebuilding them is cheap relative to the cost of repeatedly aggregating the full fact table at dashboard-refresh time.

4. Ingestion Design

`build_db.py` orchestrates the pipeline as five ordered stages (schema creation, raw ingestion, staging, warehouse, mart), each executed via `sqlite3.Connection.executescript()` against the corresponding `.sql` file, with the Python layer responsible only for CSV parsing and JSON-file loading (`dim_step`) — the actual transformation logic lives in SQL, not in Python, so it is auditable independent of the orchestration code and portable to any SQL engine with `json_extract` support (e.g., PostgreSQL with minor syntax changes).

Ingestion is **idempotent by design**: re-running `build_db.py` against the same `events.csv` drops and rebuilds every table from scratch (the script deletes any pre-existing database file before connecting), so there is no notion of a partially-applied or double-applied ingestion run. This is an appropriate design for a batch, full-refresh pipeline at this data volume; Assignment 3 (pipeline automation) is where an incremental/append-only ingestion contract — necessary once the raw dataset no longer fits comfortably in a single full rebuild — would be introduced.

5. Data Quality Framework

Data quality is treated as a first-class, executable artifact (`schema/05_data_quality_checks.sql`), not an informal assertion in documentation. Twelve checks are run after every build and reported by `build_db.py`; they are organized here by the dimension of data quality each addresses:

Dimension	Checks
Completeness	<code>raw_row_count</code> , <code>users_missing_install_event</code>
Validity	<code>staging_rejects</code> (non-zero is expected and desired — Section 5.1), <code>invalid_test_group_values</code> , <code>kpi_rates_out_of_bounds</code>
Uniqueness	<code>staging_duplicate_event_ids</code>
Referential integrity	<code>orphan_fact_users</code> (every <code>fact_user_funnel</code> row must resolve to a <code>dim_user</code> row)
Logical / funnel-order consistency	<code>completed_without_started</code> , <code>activated_without_completed</code> , <code>purchased_without_activated</code> (funnel-stage ordering must be internally consistent — a user cannot be recorded as activated without first being recorded as having completed onboarding)
Structural	<code>mart_kpi_group_count</code> (must equal 3, the number of experiment arms), <code>negative_time_to_first_lesson</code>

5.1 Verifying the Validation Logic Itself

A pipeline whose validation checks always report zero problems is ambiguous: either the data is genuinely clean, or the validation logic is not actually being exercised. To distinguish these, `build_db.py` (by default) injects a small number of deliberately malformed rows directly into `raw_events` before staging runs — a null `event_id`, a null `user_id`, a null `event_name`, a null `event_timestamp`, an out-of-domain `test_group` value, an out-of-domain `platform` value, and a duplicated `event_id` — simulating the kind of partial/corrupted delivery a real client SDK can produce. A healthy run is expected to show `staging_rejects = 6` and a `raw_row_count` exceeding `stg_events` count by 9 (6 rejected plus 3 deduplicated), which is exactly what an observed run against the generated dataset shows (Section 6). This converts “the validation logic works” from an assumption into a repeatable, observed fact.

6. Observed Results

From a run against the 6,000-user, ~129,000-event generated dataset (Assignment 1), with dirty-row injection enabled:

Check	Result
<code>raw_row_count</code>	128,983
<code>staging_rejects</code>	6
<code>staging_duplicate_event_ids</code>	0
<code>users_missing_install_event</code>	0
<code>orphan_fact_users</code>	0
<code>completed_without_started</code>	0

Check	Result
<code>activated_without_completed</code>	0
<code>purchased_without_activated</code>	0
<code>invalid_test_group_values</code>	0
<code>mart_kpi_group_count</code>	3
<code>negative_time_to_first_lesson</code>	0
<code>kpi_rates_out_of_bounds</code>	0

Arm	Users Started	Onboarding Completion Rate	Activation Rate	Paid Conversion Rate	Day-30 Return (guardrail)
A (13 steps)	2,000	0.574	0.698	0.061	0.362
B (7 steps)	2,000	0.730	0.632	0.057	0.343
C (5 steps)	2,000	0.815	0.656	0.063	0.290

All twelve checks pass at their expected value (zero, except the two intentionally non-zero counts discussed in Section 5.1), and the resulting KPI table is directionally consistent with the hypothesis in the business-understanding document: completion rate rises monotonically as step count falls, paid conversion remains approximately flat (~6%) across arms, and the Day-30 guardrail shows the expected trade-off for the shortest arm.

7. Assumptions and Limitations

- SQLite, not a production warehouse engine.** SQLite was chosen for zero-setup portability appropriate to a coursework deliverable. The SQL in `schema/` avoids SQLite-specific extensions other than `json_extract` (part of the SQL/JSON standard, also supported by PostgreSQL, MySQL 8+, and Snowflake), so migration to a production engine would primarily require changing the Python `sqlite3` connection layer, not rewriting the transformation SQL.
- Full-refresh, not incremental, ingestion.** Every run rebuilds every table from the full event history. This is appropriate at the current data volume (under 130,000 rows) but would not scale to a continuously arriving production stream; Assignment 3 is scoped to address incremental/scheduled ingestion.
- No slowly-changing-dimension (SCD) handling.** `dim_user` attributes (arm, platform) are treated as immutable for the lifetime of a user in this dataset, consistent with the “assigned once, never reassigned” randomization contract in the event-tracking plan. A production system with, e.g., platform migration (a user switching from Android to iOS) would need explicit SCD Type 2 handling, which is out of scope here.
- Single-database, single-file deployment.** No partitioning, replication, or concurrent-write handling is implemented, consistent with a batch, analyst-triggered build rather than a continuously operating service.

8. How to Run

```
python build_db.py \  
  --events-csv ../assignment1/generated_data/events.csv \  
  --stepmap-json ../assignment1/stepmap.json \  
  --db db/ab_test.db
```

Produces `db/ab_test.db` (git-ignored; regenerate locally rather than committing a large binary artifact) and prints the data-quality report and KPI/guardrail preview shown in Section 6.

See the [assignment root README](#) for the rubric-to-deliverable mapping, and the [repository root README](#) for how this assignment fits into the full four-assignment pipeline.